
Summer Research Project Report

Author:

Aouame Mohamed Yassine

Supervisor(s):

Wang Chenyang

Professor:

Pierre Dillenbourg

September 29, 2023

1 Description and set up

The repository was used on Google Colab. Changing one entry in the files (drive=0 or 1) allows to switch paths between local and drive set up. I used my personal Google account on which drive I copied the project and worked using Colab. I did not manage to handle the 80k dataset without a high ram allowance which can be unlocked by subscribing to Colab pro. Using the t4 gpu, the training does not usually exceed from 10 up to 30 minutes, except for cross-gesture and cross-subject models which can go up to 1h per fold.

1.1 Preprocessing

The preprocessing mainly focused on mapping skeleton frames (captured at 30 fps) to capacitive images (captured at 80 fps). Two strategies were used: First binning the skeleton frames and assigning the average of the corresponding capacitive images while deleting frames with too high of a jump in timestamp (which would have a very large number of capacitive images). Secondly, assigning closest capacitive images to a skeleton frame with a minimum threshold of 7 ms. Both of these strategies are described in the file "skel_cap_img_map_logic.ipynb" with an example.

The remaining of the preprocessing is done in "the generate_data.ipynb" file, it processes skeleton and capacitive images and adds other features such as the day and the native pen grip. The 7 ms closest mapping held a dataset of 81 252 samples.

To generate the data, you can run the latter file.

1.2 Training

Three helper files, data_helpers, plot_helpers and metrics_helpers are used in the training. Examples of usage can be found in model_train, data_stats and model_eval files.

Training was done in 4 different ways : train_test_split, cross_subject, cross_gesture and time_split. All models were ran on the new complete mapped data. The cross_subject is the only model that was not ran completely, I ran it on 16 folds, each fold taking about 1h to run. The loss function was also computed in 2 different ways :

First using labeling of outputs :

```
model.compile(
    optimizer=optimizer,
    loss={'skeleton': un_skeloss(mean, std),
          'gesture': 'categorical_crossentropy'},
    loss_weights={'skeleton': 1.0, 'gesture': beta},
    metrics={'skeleton': [un_epe(mean, std), un_pck(mean, std)],
             'gesture': 'accuracy'})
```

However the scales of gesture loss and skeleton loss being two disparate, I preferred to concatenate both outputs in the "defined_model.ipynb" file to get access to the range of

values.

```
x_gesture = Dense(4, activation='softmax', name='gesture')(fc)
x_skeleton = Dense(63, activation="linear", name='skeleton')(fc)
concat_output = tf.keras.layers.Concatenate(axis=-1,
        trainable=False)([x_skeleton, x_gesture])
model = Model(inputs=inputs, outputs=[concat_output])
```

I did not manage to normalize both losses, the computation looks numerically unstable. It can be found in the `w_loss` function of the `metrics_helper.py` file. I ran the time models on full gesture (meaning 1 for gesture loss weight and 0 for skeleton loss) but the model seems to over fit very quickly. A less complex model would be more suited solely for this task, and fixing the loss normalization problem would be the better solution.

The `defined_model.py` file describes 4 models. Two regular 2d and 3d models, and two deeper models which use more regularization and dropout.

For all trained models, a learning rate of $1e-4$ was used for the Adam Optimizer, as well as callbacks to reduce learning rate by a factor of 0.5 with a patience of 5 and to perform an early stop with patience of 10.

A good example to get familiar with the code would be section `"3d_models"` and `"2d_models"` subsections from `"Time model"` section in `"model_train.ipynb"` which use the data loading functions for different needs.

1.3 Evaluation and Visualization

Examples of evaluation on test data can be found after each training cells in the `"model_train.ipynb"` file, as well as in the `"data_stats.ipynb"` and `"model_eval.ipynb"` file. The latter hold examples of use for the plotting functions which help visualize distributions, as well as skeleton hands in 2d/3d and capacitive images heatmaps for some special cases (for example visualizing the worst predictions of the models versus the best).

2 Results

In this section, we present the results for our models. All of these models were trained using z-score normalization on capacitive images over all data samples.

Skeleton coordinates are normalized this way :

First we subtract the wrist coordinates to all joints coordinates to get the relative coordinates. Then we make a choice: either keep the original wrist coordinates or label them as 0, which we refer to as "keeping_wrist" and "without_wrist".

The coordinates are then z-score normalized either over all joints, over the available axis or over all data samples, which we refer to as "joint", "axis", and "all".

The 2d models were trained by removing one axis. We tried the possible combinations of removal of axis and z-score normalization.

Table 1: Summary results of variations of 2d models

Model	Test EPE	Val EPE	Test G acc	Test G fl
keep_wrist removing x "all"	99.85	70.35	0.37	0.17
keep_wrist removing x "joint"	100.34	70.80	0.31	0.19
keep_wrist removing x "axis"	100.85	73.04	0.34	0.15
keep_wrist removing y "all"	136.50	104.47	0.34	0.58
keep_wrist removing y "joint"	136.17	101.53	0.28	0.09
keep_wrist removing y "axis"	137.26	101.92	0.31	0.13
keep_wrist removing z "all"	127.30	98.69	0.41	0.10
keep_wrist removing z "joint"	128.84	97.17	0.37	0.09
keep_wrist removing z "axis"	127.85	99.18	0.35	0.17
without_wrist removing x "all"	88.56	64.79	0.36	0.14
without_wrist removing x "joint"	90.71	63.52	0.38	0.22
without_wrist removing x "axis"	88.80	64.75	0.41	0.25
without_wrist removing y "all"	117.60	94.35	0.28	0.01
without_wrist removing y "joint"	117.27	88.97	0.32	0.07
without_wrist removing y "axis"	113.18	87.99	0.36	0.20
without_wrist removing z "all"	113.81	80.87	0.35	0.05
without_wrist removing z "joint"	111.89	87.78	0.33	0.06
without_wrist removing z "axis"	111.19	85.89	0.36	0.20

Hence, joint z-score normalization seems to be the best skeleton normalization, alongside with removing axis 'x' for 2d models.

From now on all models are trained using z-score normalized capacitive images, joint z-score normalized skeletons. 2d models are obtained by removing the 'x' axis.

The cross-gesture and cross-subject are trained without filtering the capacitive images (besides removing empty ones), and using a 3d model.

Table 2: Summary results of cross gesture and cross subject 3d models

Model	Test EPE	Test G acc	Test G f1
cross DQ	128.44	0	0
cross DT	130.90	0	0
cross LQ	117.11	0	0
cross LT	128.57	0	0
cross subject 1	216.47	0.20	0.00
cross subject 2	113.57	0.34	0.31
cross subject 5	77.17	0.68	0.66
cross subject 10	133.71	0.24	0.20
cross subject 11	99.80	0.16	0.14
cross subject 12	109.23	0.55	0.53
cross subject 13	144.62	0.38	0.35
cross subject 14	86.70	0.23	0.21
cross subject 16	209.90	0.19	0.16
cross subject 17	442.21	0.60	0.58
cross subject 18	134.36	0.25	0.21
cross subject 19	166.75	0.23	0.20
cross subject 20	158.44	0.20	0.17
cross subject 21	72.26	0.29	0.28
cross subject 22	119.65	0.49	0.49
cross subject 24	96.08	0.29	0.26

Table 3: Results of gesture stratified train test validation split 3d model

Model	Test EPE	Val EPE	Test G acc	Test G f1
train_test_split	21.8	21.5	0.90	0.90

Concerning the time models, the models cover 2d and 3d as well as keeping and without wrist. The training is done on 60% of the data and is referred as "base" for both 2d and 3d models. Trying to minimize overfitting, we also tried regularized deeper models, as well as training on 80% of data. These reg models get to a comparable val loss to base model, while the training loss significantly stays higher, meaning less over fitting.

We also decided to perform filtering over our samples after finding some samples with very light measurements in their capacitive images, meaning very little contact to the tablet. The filtering method rules out sample based on their number of non zero pixels with respect to the mean number of non zeroes pixels in the whole dataset. For instance, the filter $\pm$ std methods keeps capacitive images with n non zeroes pixels if $\text{mean} - 2 \cdot \text{std}$ smaller than n and n greater than $\text{mean} + 2 \cdot \text{std}$, while the filter - 5 does the same for n greater than $\text{mean} - 5$ (only a lower bound).

Table 4: Numbers of data points after filtering

filter	Number of samples
base	80 960
filter mean +- 2 std	78 910
filter mean - std	68 619
filter mean +- std	57 303
filter mean - 15	49 154
filter mean - 5	40 762
filter mean	36 943

Table 5: Summary results of 2d time models, keeping wrist

Model	Test EPE	Val EPE	Test G acc	Test G f1
base	100.49	70.48	0.38	0.21
base 80% data	100.00	85.16	0.40	0.16
deeper base regularized	100.00	72.60	0.33	0.21
deeper base regularized 80% data	98.22	77.51	0.48	0.26
base full gesture	-	-	0.40	0.39
filter mean +- 2 std	100.31	70.50	0.36	0.27
filter mean +- std	101.31	67.18	0.40	0.28
filter mean - std	98.70	67.94	0.37	0.14
filter mean - 15	93.87	61.91	0.45	0.37
filter mean - 5	93.25	60.87	0.43	0.31
filter mean	90.56	60.26	0.42	0.36
deeper regularized filter mean	88.95	59.18	0.46	0.24
filter mean 80% data	95.69	60.79	0.56	0.48
deeper regularized filter mean 80% data	92.05	61.89	0.45	0.30

Some points to explore :

- The worst and best prediction on time models are shown with capacitive images heatmaps and hand skeleton visualizations in the "model_eval.ipynb" file. A pattern could be derived.
- The filtering on non dense capacitive images samples enhanced the results, a better filtering could clean better the data.
- Some subjects (subject 5, subject 21, subject 17) have good EPE and gesture accuracy, looking at the reasons could explain the models' flaws.

Table 6: Summary results of 2d time models, without wrist

Model	Test EPE	Val EPE	Test G acc	Test G f1
base	88.55	64.30	0.36	0.10
base 80% data	89.38	77.23	0.39	0.14
deeper base regularized	87.81	65.60	0.32	0.16
deeper base regularized 80% data	89.00	73.19	0.43	0.27
base full gesture	-	-	0.33	0.31
filter mean +- 2 std	89.67	64.15	0.34	0.22
filter mean +- std	91.00	62.71	0.39	0.14
filter mean - std	87.90	63.01	0.32	0.05
filter mean - 15	87.67	58.88	0.44	0.33
filter mean - 5	85.64	58.02	0.40	0.23
filter mean	82.93	56.51	0.40	0.13
deeper regularized filter mean	85.32	58.19	0.37	0.17
filter mean 80% data	88.54	57.50	0.49	0.42
deeper regularized filter mean 80% data	86.31	57.19	0.45	0.29

Table 7: Summary results of 3d time models, keeping wrist

Model	Test EPE	Val EPE	Test G acc	Test G f1
train_test_split	21.8	21.5	0.90	0.90
base	151.00	112.00	0.33	0.12
base 80% data	157.73	111.32	0.39	0.09
deeper base regularized	148.74	112.00	0.35	0.10
deeper base regularized 80% data	152.30	118.22	0.41	0.19
base full gesture	-	-	0.33	0.30
filter mean +- 2 std	147.67	113.00	0.35	0.10
filter mean +- std	150.13	107.10	0.42	0.18
filter mean - std	145.64	107.00	0.38	0.15
filter mean - 5	137.72	97.87	0.37	0.14
filter mean	137.40	95.15	0.42	0.27
deeper regularized filter mean	136.92	94.30	0.33	0.17
filter mean 80% data	142.90	96.79	0.44	0.26
deeper regularized filter mean 80% data	136.25	94.06	0.48	0.31

Table 8: Summary results of 3d time models, without wrist

Model	Test EPE	Val EPE	Test G acc	Test G f1
base	132.14	101.00	0.34	0.05
base 80% data	130.15	113.09	0.35	0.13
deeper base regularized	128.00	98.67	0.33	0.11
deeper base regularized 80% data	131.20	110.71	0.42	0.10
base full gesture	-	-	0.37	0.34
filter mean +- 2 std	131.94	100.80	0.33	0.13
filter mean +- std	131.30	96.47	0.41	0.13
filter mean - std	131.74	93.97	0.38	0.11
filter mean - 5	122.13	90.66	0.41	0.18
filter mean	119.67	86.49	0.41	0.16
deeper regularized filter mean	117.25	86.77	0.36	0.09
filter mean 80% data	123.88	83.36	0.43	0.23
deeper regularized filter mean 80% data	120.42	81.30	0.42	0.11